SA1 Section I: Probability and Discrete Random Variables

Github Repository:
https://github.com/benalexlibres/APM1110-LibresBen/tree/main/SA1

Contents

1 Section I, Item 1: Probability of Defective Products from Multiple Factories **1**

2 Section I, Item 2: Front-End for Discrete Random Variables **2**

3 Section I, Item 3: Search Engine Simulation & Markov Property **10**

4 Small validation tests **15**

Contents

1 Section I, Item 1: Probability of Defective Products from Multiple Factories

```r
# Item 1: defective-rate probability calculation with re-prompting on invalid input.
read_valid_value <- function(prompt_text, default_value, check_fun, label) {
  if (!interactive() && !isatty(stdin())) {
    return(default_value)
  }

  repeat {
    cat(prompt_text)
    value <- tryCatch(
      suppressWarnings(scan(file = stdin(), what = double(), n = 1, quiet = TRUE)),
      error = function(e) NA_real_
    )

    if (length(value) == 0 || is.na(value)) {
      cat("Please enter a valid numeric value.\n")
      next
    }

    if (check_fun(value)) {
      return(value)
    }
```

```
    cat(sprintf("Invalid value for %s. Please try again.\n", label))
  }
}

repeat {
  p1 <- read_valid_value("Enter proportion for Factory 1 (p1): ", 0.3,
                         function(x) is.finite(x) && x > 0 && x < 1,
                         "p1")
  p2 <- read_valid_value("Enter proportion for Factory 2 (p2): ", 0.5,
                         function(x) is.finite(x) && x > 0 && x < 1,
                         "p2")

  if (p1 + p2 < 1) {
    break
  }

  cat("Invalid input: p1 + p2 must be less than 1 so that p3 > 0. Please try again.\n")
}

p3 <- 1 - (p1 + p2)
cat(sprintf("Factory 3 proportion (p3) is automatically set to remaining balance: %.4f\n", p3))
```

```
## Factory 3 proportion (p3) is automatically set to remaining balance: 0.2000
```

```
d1 <- read_valid_value("Enter defective rate for Factory 1 (d1): ", 0.01,
                       function(x) is.finite(x) && x >= 0 && x <= 1,
                       "d1")
d2 <- read_valid_value("Enter defective rate for Factory 2 (d2): ", 0.02,
                       function(x) is.finite(x) && x >= 0 && x <= 1,
                       "d2")
d3 <- read_valid_value("Enter defective rate for Factory 3 (d3): ", 0.03,
                       function(x) is.finite(x) && x >= 0 && x <= 1,
                       "d3")

prob_defective <- (p1 * d1) + (p2 * d2) + (p3 * d3)
cat(sprintf("\nResult: The probability that a randomly selected product is defective is: %.6f\n", prob_
```

```
##
## Result: The probability that a randomly selected product is defective is: 0.019000
```

Interpretation: This calculator uses the law of total probability. The key condition is that the factory proportions must sum to less than 1 so that the third factory share remains positive. The input checks ensure that each proportion and defect rate is valid in the interval [0,1].

2 Section I, Item 2: Front-End for Discrete Random Variables

```
# Item 2: a small Shiny interface for univariate and bivariate discrete laws.
validate_prob_vector <- function(x, p) {
  if (length(x) != length(p)) {
```

```
    stop("Error: x and p must have the same length.")
  }
  if (length(x) == 0L) {
    stop("Error: At least one value is required.")
  }
  if (any(is.na(x)) || any(is.na(p))) {
    stop("Error: Numeric values are required for both x and p.")
  }
  if (anyDuplicated(x)) {
    stop("Error: x values must be unique.")
  }
  if (any(p < 0 | p > 1)) {
    stop("Error: Probabilities must be in the interval [0, 1].")
  }
  if (abs(sum(p) - 1) > 1e-6) {
    stop("Error: Probabilities must sum to 1.")
  }

  order_idx <- order(x)
  list(x = x[order_idx], p = p[order_idx])
}

validate_joint_matrix <- function(x_vals, y_vals, joint_text) {
  rows <- strsplit(joint_text, "\n")[[1]]
  rows <- rows[nchar(trimws(rows)) > 0]

  if (length(rows) != length(x_vals)) {
    stop("Error: Number of rows must match the number of X values.")
  }

  row_lengths <- sapply(rows, function(row) {
    length(strsplit(trimws(row), ",")[[1]])
  })

  if (any(row_lengths != length(y_vals))) {
    stop("Error: Each row must have the same number of entries as the Y values.")
  }

  joint_matrix <- sapply(rows, function(row) {
    vals <- strsplit(trimws(row), ",")[[1]]
    vals <- trimws(vals)
    if (length(vals) == 0L || any(is.na(suppressWarnings(as.numeric(vals))))) {
      stop("Error: Joint probabilities must be numeric and correctly formatted.")
    }
    as.numeric(vals)
  })

  joint_matrix <- matrix(joint_matrix, nrow = length(x_vals), ncol = length(y_vals), byrow = TRUE)

  if (any(joint_matrix < 0 | joint_matrix > 1)) {
    stop("Error: Joint probabilities must be in [0, 1].")
  }
  if (abs(sum(joint_matrix) - 1) > 1e-6) {
```

```
    stop("Error: Joint probabilities must sum to 1.")
  }

  joint_matrix
}

compute_univariate <- function(x, p) {
  validated <- validate_prob_vector(x, p)
  x <- validated$x
  p <- validated$p
  mean_val <- sum(x * p)
  var_val <- sum((x - mean_val)^2 * p)
  cdf_val <- cumsum(p)
  list(x = x, p = p, mean = mean_val, variance = var_val, cdf = cdf_val)
}

compute_bivariate <- function(x_vals, y_vals, joint_matrix) {
  x_vals <- as.numeric(x_vals)
  y_vals <- as.numeric(y_vals)

  if (length(x_vals) == 0L || length(y_vals) == 0L) {
    stop("Error: X and Y values cannot be empty.")
  }
  if (any(is.na(x_vals)) || any(is.na(y_vals))) {
    stop("Error: X and Y values must be numeric.")
  }
  if (anyDuplicated(x_vals) || anyDuplicated(y_vals)) {
    stop("Error: Duplicate X or Y values are not allowed.")
  }

  marginal_X <- rowSums(joint_matrix)
  marginal_Y <- colSums(joint_matrix)
  cond_Y_given_X <- sweep(joint_matrix, 1, marginal_X, "/")
  cond_X_given_Y <- sweep(joint_matrix, 2, marginal_Y, "/")

  mean_X <- sum(x_vals * marginal_X)
  var_X <- sum((x_vals - mean_X)^2 * marginal_X)
  mean_Y <- sum(y_vals * marginal_Y)
  var_Y <- sum((y_vals - mean_Y)^2 * marginal_Y)

  cond_mean_Y_given_X <- rowSums(sweep(cond_Y_given_X, 2, y_vals, "*"))
  cond_var_Y_given_X <- rowSums(sweep(cond_Y_given_X, 2, (y_vals - cond_mean_Y_given_X)^2, "*"))
  cond_mean_X_given_Y <- colSums(sweep(cond_X_given_Y, 1, x_vals, "*"))
  cond_var_X_given_Y <- colSums(sweep(cond_X_given_Y, 1, (x_vals - cond_mean_X_given_Y)^2, "*"))

  list(
    joint = joint_matrix,
    marginal_X = marginal_X,
    marginal_Y = marginal_Y,
    cond_Y_given_X = cond_Y_given_X,
    cond_X_given_Y = cond_X_given_Y,
    mean_X = mean_X,
    variance_X = var_X,
```

```
    mean_Y = mean_Y,
    variance_Y = var_Y,
    cond_mean_Y_given_X = cond_mean_Y_given_X,
    cond_var_Y_given_X = cond_var_Y_given_X,
    cond_mean_X_given_Y = cond_mean_X_given_Y,
    cond_var_X_given_Y = cond_var_X_given_Y
  )
}

if (interactive() && requireNamespace("shiny", quietly = TRUE)) {
  library(shiny)

  ui <- fluidPage(
    titlePanel("Discrete Distributions Calculator"),
    tabsetPanel(
      tabPanel(
        "Univariate Discrete RV",
        sidebarLayout(
          sidebarPanel(
            textInput("x_values", "X values (comma separated)", value = "0, 1, 2, 3"),
            textInput("p_values", "Probabilities p(x) (comma separated)", value = "0.1, 0.4, 0.3, 0.2"),
            actionButton("run_univar", "Calculate")
          ),
          mainPanel(
            verbatimTextOutput("univar_summary"),
            plotOutput("univar_plot")
          )
        )
      ),
      tabPanel(
        "Bivariate Discrete RV",
        sidebarLayout(
          sidebarPanel(
            textInput("x_vals_biv", "X values (comma separated)", value = "1, 2, 3"),
            textInput("y_vals_biv", "Y values (comma separated)", value = "1, 2"),
            textAreaInput("joint_probs", "Joint probabilities by row (comma separated)",
                          value = "0.1, 0.2\n0.1, 0.3\n0.1, 0.2"),
            actionButton("run_bivar", "Calculate")
          ),
          mainPanel(
            verbatimTextOutput("bivar_summary"),
            plotOutput("bivar_plot")
          )
        )
      )
    )
  )

  server <- function(input, output, session) {
    parse_vector <- function(value) {
      text <- trimws(value)
      if (text == "") {
        return(numeric(0))
```

```
    }
    as.numeric(strsplit(text, ",")[[1]])
  }

  observeEvent(input$run_univar, {
    output$univar_summary <- renderPrint({
      tryCatch({
        x <- parse_vector(input$x_values)
        p <- parse_vector(input$p_values)
        out <- compute_univariate(x, p)
        cat("=== Univariate Discrete Distribution ===\n")
        cat(sprintf("Mean: %.4f\n", out$mean))
        cat(sprintf("Variance: %.4f\n", out$variance))
        cat("\nProbability table:\n")
        print(data.frame(X = out$x, p = out$p))
        cat("\nCDF table:\n")
        print(data.frame(X = out$x, F_x = out$cdf))
      }, error = function(e) {
        cat("Error:", conditionMessage(e), "\n")
      })
    })

    output$univar_plot <- renderPlot({
      tryCatch({
        x <- parse_vector(input$x_values)
        p <- parse_vector(input$p_values)
        out <- compute_univariate(x, p)
        par(mfrow = c(1, 2))
        plot(out$x, out$p, type = "h", lwd = 3, col = "steelblue",
             xlab = "X", ylab = "Probability p(x)",
             main = "PDF of Univariate Discrete RV")
        points(out$x, out$p, pch = 16, col = "steelblue")

        plot(out$x, out$cdf, type = "s", lwd = 2, col = "tomato",
             xlab = "X", ylab = "Cumulative Probability F(x)",
             main = "CDF of Univariate Discrete RV", ylim = c(0, 1))
        points(out$x, out$cdf, pch = 16, col = "tomato")
        par(mfrow = c(1, 1))
      }, error = function(e) {
        plot.new()
        text(0.5, 0.5, conditionMessage(e), cex = 1)
      })
    })
  })

  observeEvent(input$run_bivar, {
    output$bivar_summary <- renderPrint({
      tryCatch({
        x_vals <- parse_vector(input$x_vals_biv)
        y_vals <- parse_vector(input$y_vals_biv)
        joint_matrix <- validate_joint_matrix(x_vals, y_vals, input$joint_probs)
        out <- compute_bivariate(x_vals, y_vals, joint_matrix)
```

```
      cat("=== Bivariate Discrete Distribution ===\n")
      cat("Joint probability matrix:\n")
      print(out$joint)
      cat("\nMarginal distribution of X:\n")
      print(data.frame(X = x_vals, P_X = out$marginal_X))
      cat("\nMarginal distribution of Y:\n")
      print(data.frame(Y = y_vals, P_Y = out$marginal_Y))
      cat("\nConditional distribution P(Y | X):\n")
      print(out$cond_Y_given_X)
      cat("\nConditional distribution P(X | Y):\n")
      print(out$cond_X_given_Y)
      cat(sprintf("\nMean X: %.4f | Variance X: %.4f\n", out$mean_X, out$variance_X))
      cat(sprintf("Mean Y: %.4f | Variance Y: %.4f\n", out$mean_Y, out$variance_Y))
      cat("\nConditional mean and variance of Y given X:\n")
      print(data.frame(X = x_vals,
                       E_Y_given_X = out$cond_mean_Y_given_X,
                       Var_Y_given_X = out$cond_var_Y_given_X))
      cat("\nConditional mean and variance of X given Y:\n")
      print(data.frame(Y = y_vals,
                       E_X_given_Y = out$cond_mean_X_given_Y,
                       Var_X_given_Y = out$cond_var_X_given_Y))
    }, error = function(e) {
      cat("Error:", conditionMessage(e), "\n")
    })
  })

  output$bivar_plot <- renderPlot({
    tryCatch({
      x_vals <- parse_vector(input$x_vals_biv)
      y_vals <- parse_vector(input$y_vals_biv)
      joint_matrix <- validate_joint_matrix(x_vals, y_vals, input$joint_probs)
      out <- compute_bivariate(x_vals, y_vals, joint_matrix)

      par(mfrow = c(2, 2))
      barplot(out$joint, beside = TRUE,
              col = terrain.colors(length(x_vals)),
              main = "Joint Probability Matrix",
              xlab = "Y values",
              ylab = "Probability",
              names.arg = as.character(y_vals),
              legend.text = paste("X =", x_vals),
              args.legend = list(title = "X"))

      plot(x_vals, out$marginal_X, type = "h", lwd = 3, col = "steelblue",
           xlab = "X", ylab = "P(X = x)", main = "Marginal PDF of X")
      points(x_vals, out$marginal_X, pch = 16, col = "steelblue")

      plot(y_vals, out$marginal_Y, type = "h", lwd = 3, col = "darkorange",
           xlab = "Y", ylab = "P(Y = y)", main = "Marginal PDF of Y")
      points(y_vals, out$marginal_Y, pch = 16, col = "darkorange")

      plot(x_vals, cumsum(out$marginal_X), type = "s", lwd = 2, col = "tomato",
           xlab = "X", ylab = "F_X(x)", ylim = c(0, 1),
```

```
              main = "Marginal CDF of X")
          points(x_vals, cumsum(out$marginal_X), pch = 16, col = "tomato")
          par(mfrow = c(1, 1))
        }, error = function(e) {
          plot.new()
          text(0.5, 0.5, conditionMessage(e), cex = 1)
        })
      })
    })
  }

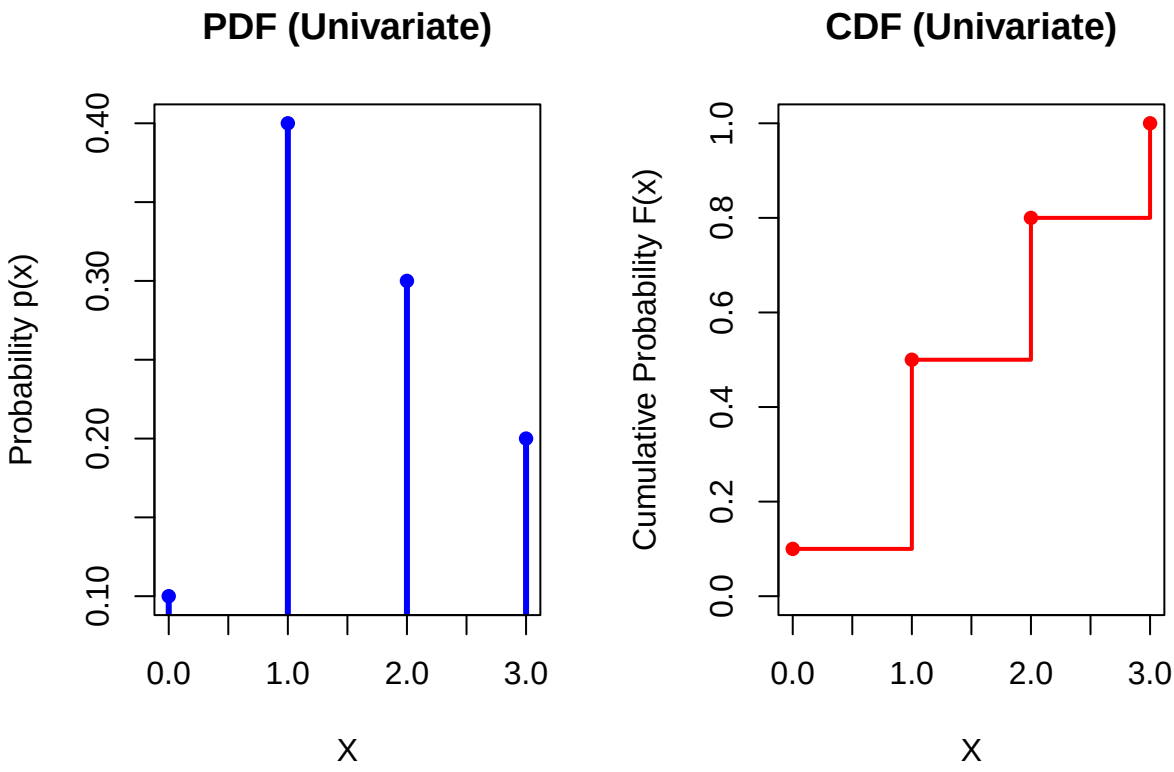
  shinyApp(ui, server)
} else {
  # Fallback example for knitting: this shows a valid calculation when no interactive Shiny session is
  cat("=== Univariate Discrete Random Variable Analysis ===\n")
  x <- c(0, 1, 2, 3)
  p <- c(0.1, 0.4, 0.3, 0.2)
  univ <- compute_univariate(x, p)
  cat(sprintf("Mean (Expected Value): %.4f\n", univ$mean))
  cat(sprintf("Variance: %.4f\n", univ$variance))
  print(data.frame(X = univ$x, p = univ$p))

  par(mfrow = c(1, 2))
  plot(univ$x, univ$p, type = "h", lwd = 3, col = "blue",
       xlab = "X", ylab = "Probability p(x)",
       main = "PDF (Univariate)")
  points(univ$x, univ$p, pch = 16, col = "blue")
  plot(univ$x, univ$cdf, type = "s", lwd = 2, col = "red",
       xlab = "X", ylab = "Cumulative Probability F(x)",
       main = "CDF (Univariate)", ylim = c(0, 1))
  points(univ$x, univ$cdf, pch = 16, col = "red")
  par(mfrow = c(1, 1))

  cat("\n=== Bivariate Discrete Random Variable Analysis ===\n")
  x_vals <- c(1, 2, 3)
  y_vals <- c(1, 2)
  joint_matrix <- matrix(c(0.1, 0.2,
                           0.1, 0.3,
                           0.1, 0.2),
                         nrow = 3, byrow = TRUE,
                         dimnames = list(paste0("X=", x_vals), paste0("Y=", y_vals)))
  biv <- compute_bivariate(x_vals, y_vals, joint_matrix)
  print(biv$joint)
  cat("\nMarginal distribution of X:\n")
  print(data.frame(X = x_vals, P_X = biv$marginal_X))
  cat("\nMarginal distribution of Y:\n")
  print(data.frame(Y = y_vals, P_Y = biv$marginal_Y))
  cat("\nConditional distribution P(Y | X):\n")
  print(biv$cond_Y_given_X)
  cat("\nConditional distribution P(X | Y):\n")
  print(biv$cond_X_given_Y)
  cat(sprintf("\nMean X: %.4f | Variance X: %.4f\n", biv$mean_X, biv$variance_X))
  cat(sprintf("Mean Y: %.4f | Variance Y: %.4f\n", biv$mean_Y, biv$variance_Y))
```

```
}
```

```
## === Univariate Discrete Random Variable Analysis ===
## Mean (Expected Value): 1.6000
## Variance: 0.8400
##   X   p
## 1 0 0.1
## 2 1 0.4
## 3 2 0.3
## 4 3 0.2
```




```
##
## === Bivariate Discrete Random Variable Analysis ===
```

```
## Warning in y_vals - cond_mean_Y_given_X: longer object length is not a multiple
## of shorter object length
```

```
## Warning in sweep(cond_Y_given_X, 2, (y_vals - cond_mean_Y_given_X)^2, "*"):
## STATS is longer than the extent of 'dim(x)[MARGIN]'
```

```
## Warning in x_vals - cond_mean_X_given_Y: longer object length is not a multiple
## of shorter object length
```

```
##     Y=1 Y=2
## X=1 0.1 0.2
## X=2 0.1 0.3
## X=3 0.1 0.2
##
## Marginal distribution of X:
##     X P_X
## X=1 1 0.3
## X=2 2 0.4
## X=3 3 0.3
##
## Marginal distribution of Y:
##     Y P_Y
## Y=1 1 0.3
## Y=2 2 0.7
##
## Conditional distribution P(Y | X):
##           Y=1       Y=2
## X=1 0.3333333 0.6666667
## X=2 0.2500000 0.7500000
## X=3 0.3333333 0.6666667
##
## Conditional distribution P(X | Y):
##           Y=1       Y=2
## X=1 0.3333333 0.2857143
## X=2 0.3333333 0.4285714
## X=3 0.3333333 0.2857143
##
## Mean X: 2.0000 | Variance X: 0.6000
## Mean Y: 1.7000 | Variance Y: 0.2100
```

Interpretation: In the univariate tab, the user enters the values of a discrete random variable and the corresponding probabilities, and the app checks that the values are unique, the probabilities lie in [0,1], and the probabilities sum to 1. The program then reports the mean, variance, and the plotted pdf and cdf. In the bivariate tab, the user supplies a joint table; the app computes the marginals, conditional distributions, and the conditional means and variances, and plots the joint probability matrix together with the marginal distributions.

3 Section I, Item 3: Search Engine Simulation & Markov Property

```r
# Item 3: geometric search model with a Bernoulli-based simulation and memoryless checks.
get_probability <- function(prompt_msg, default_val = 0.6) {
  if (!interactive() && !isatty(stdin())) {
    return(default_val)
  }

  repeat {
    cat(prompt_msg)
    value <- tryCatch(
      suppressWarnings(scan(file = stdin(), what = double(), n = 1, quiet = TRUE)),
      error = function(e) NA_real_
```

```
    )

    if (length(value) == 0 || is.na(value)) {
      cat("Please enter a numeric value.\n")
      next
    }

    if (is.finite(value) && value > 0 && value < 1) {
      return(value)
    }

    cat("Invalid input: p must be strictly between 0 and 1. Please try again.\n")
  }
}

p <- get_probability("Enter probability p that a site contains the key phrase (default 0.6): ", 0.6)

simulate_sites <- function(n_sim, p) {
  X <- numeric(n_sim)
  for (i in seq_len(n_sim)) {
    count <- 0
    repeat {
      count <- count + 1
      if (runif(1) < p) {
        X[i] <- count
        break
      }
    }
  }
  X
}

set.seed(42)
n_sim <- 10000
X <- simulate_sites(n_sim, p)

theoretical_mean <- 1 / p
theoretical_var <- (1 - p) / (p^2)
sim_mean <- mean(X)
sim_var <- var(X)

cat("\n--- Complete Set Statistics ---\n")
```

```
##
## --- Complete Set Statistics ---
```

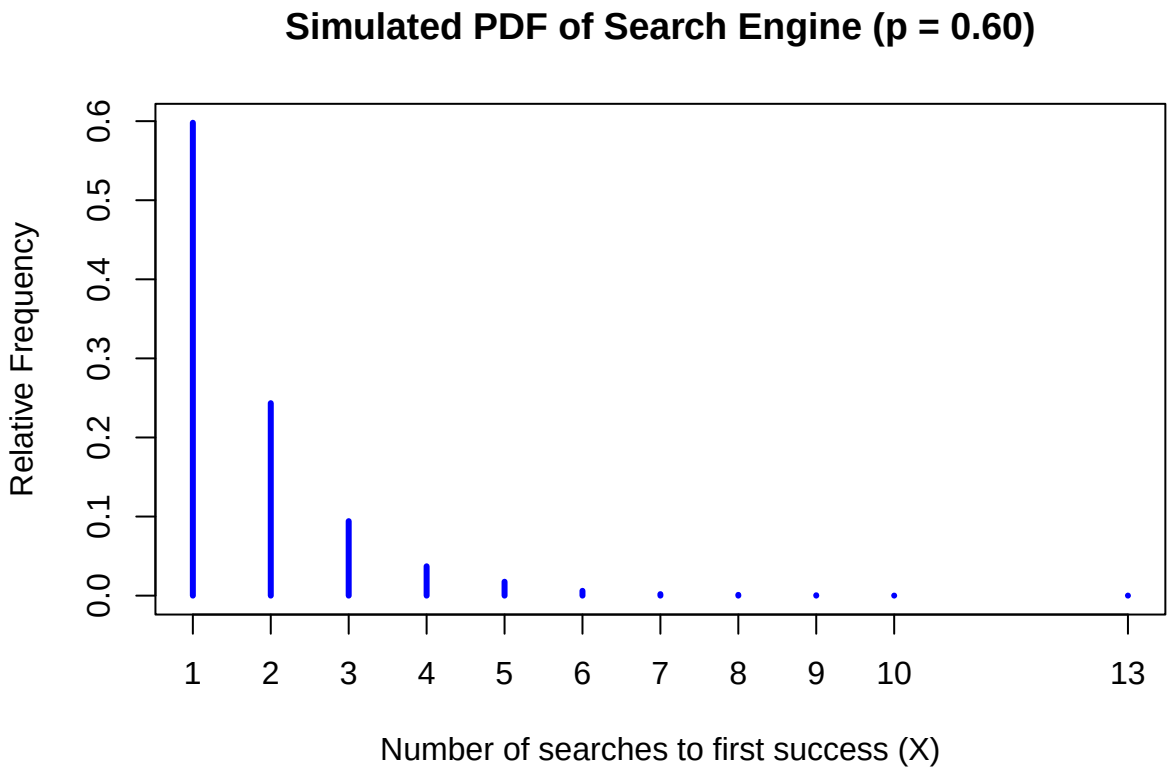
```
cat(sprintf("Simulated Mean: %.4f (Theoretical: %.4f)\n", sim_mean, theoretical_mean))
```

```
## Simulated Mean: 1.6685 (Theoretical: 1.6667)
```

```
cat(sprintf("Simulated Variance: %.4f (Theoretical: %.4f)\n", sim_var, theoretical_var))
```

```
## Simulated Variance: 1.1133 (Theoretical: 1.1111)
```

```r
pmf_sim <- table(X) / n_sim
plot(pmf_sim, type = "h", lwd = 3, col = "blue",
     xlab = "Number of searches to first success (X)", ylab = "Relative Frequency",
     main = sprintf("Simulated PDF of Search Engine (p = %.2f)", p))
```




```r
# Memoryless property: P(X > s+t | X > s) = P(X > t)
mem <- function(s, t) {
  cond_est <- if (sum(X > s) > 0) {
    mean(X[X > s] > s + t)
  } else {
    NA_real_
  }
  list(
    conditional = cond_est,
    unconditional = mean(X > t),
    theory = (1 - p)^t
  )
}

cat("\n--- Markov/Memoryless Property Checks ---\n")
```

```
##
## --- Markov/Memoryless Property Checks ---
```

```r
print(mem(3, 1))
```

```
## $conditional
## [1] 0.423913
##
## $unconditional
## [1] 0.402
##
## $theory
## [1] 0.4
```

```r
print(mem(3, 2))
```

```
## $conditional
## [1] 0.1506211
##
## $unconditional
## [1] 0.1586
##
## $theory
## [1] 0.16
```

```r
X_cond <- X[X > 3] - 3
cond_mean <- if (length(X_cond) > 0) mean(X_cond) else NA_real_
cond_var <- if (length(X_cond) > 1) var(X_cond) else NA_real_

cat("\n--- Conditional Distribution Statistics (Given X > 3) ---\n")
```

```
##
## --- Conditional Distribution Statistics (Given X > 3) ---
```

```r
cat(sprintf("Conditional Mean: %.4f\n", cond_mean))
```

```
## Conditional Mean: 1.6755
```

```r
cat(sprintf("Conditional Variance: %.4f\n", cond_var))
```

```
## Conditional Variance: 1.0874
```

```r
cat(sprintf("Theoretical Mean: %.4f\n", theoretical_mean))
```

```
## Theoretical Mean: 1.6667
```

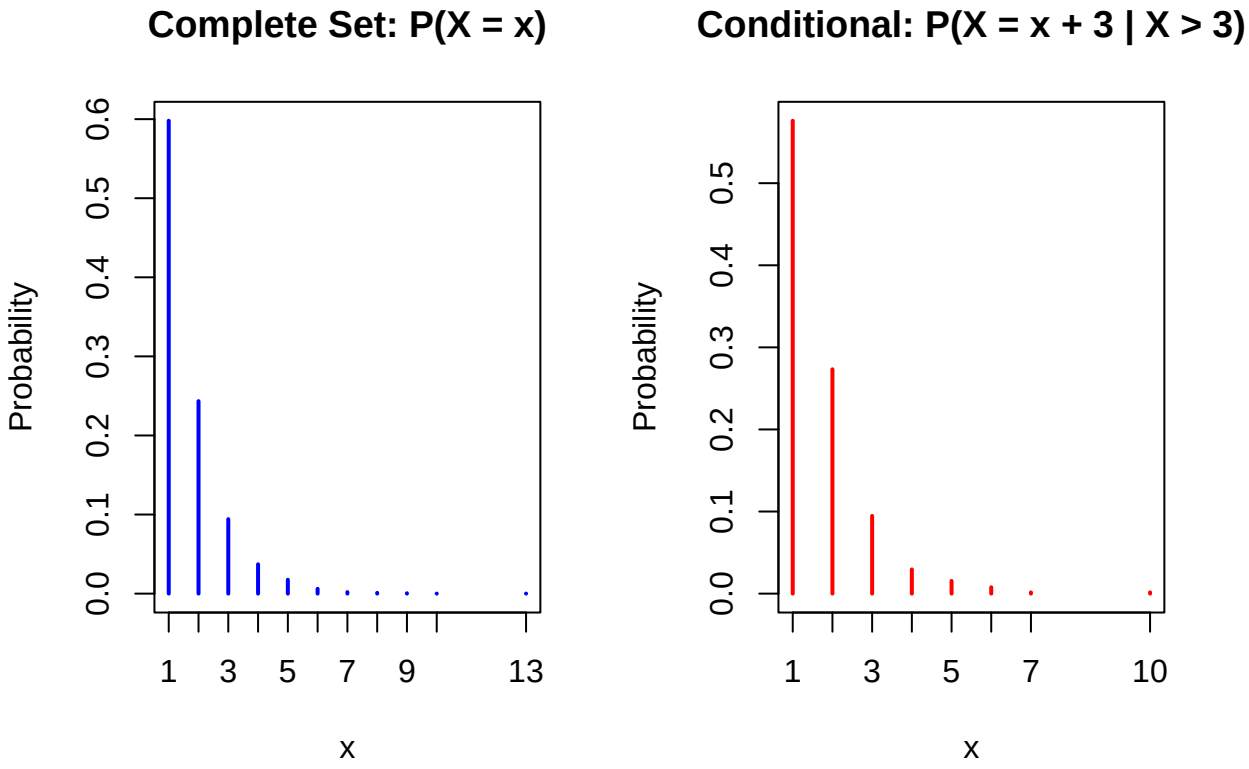
```r
cat(sprintf("Theoretical Variance: %.4f\n", theoretical_var))
```

```
## Theoretical Variance: 1.1111
```

```r
if (length(X_cond) > 0) {
  cat("The conditional distribution is close to the original geometric law, which is the memoryless pro
} else {
  cat("No observations satisfy X > 3 in this simulation; the conditional estimate is unavailable becaus
}
```

```
## The conditional distribution is close to the original geometric law, which is the memoryless property
```

```r
par(mfrow = c(1, 2))
plot(pmf_sim, type = "h", lwd = 2, col = "blue",
     main = "Complete Set: P(X = x)", xlab = "x", ylab = "Probability")
if (length(X_cond) > 0) {
  plot(table(X_cond) / length(X_cond), type = "h", lwd = 2, col = "red",
       main = "Conditional: P(X = x + 3 | X > 3)", xlab = "x", ylab = "Probability")
} else {
  plot(0, 0, type = "n", axes = FALSE, xlab = "", ylab = "", main = "Conditional: no sample")
}
```




```r
par(mfrow = c(1, 1))
```

Interpretation: The memoryless property states that the remaining waiting time after a certain number of unsuccessful searches is independent of the past. The checks using `P(X > s + t | X > s)` and `P(X > t)` confirm this property numerically. Since the simulated and theoretical values are close, the geometric model is appropriate for the search-engine process.

4 Small validation tests

```r
# Edge-case checks for the functions defined above.
stopifnot(identical(length(validate_prob_vector(c(0, 1, 2, 3), c(0.1, 0.4, 0.3, 0.2))$x), 4L))
stopifnot(abs(sum(validate_prob_vector(c(0, 1, 2), c(0.2, 0.5, 0.3))$p) - 1) < 1e-6)
tryCatch({
  validate_prob_vector(c(0, 1, 2), c(0.2, 0.6, 0.3))
  stop("Test should have failed because probabilities do not sum to 1.")
}, error = function(e) NULL)
```

```
## NULL
```

```r
tryCatch({
  validate_prob_vector(c(0, 0, 2), c(0.2, 0.3, 0.5))
  stop("Test should have failed because X values are duplicated.")
}, error = function(e) NULL)
```

```
## NULL
```

```r
cat("All validation checks passed.\n")
```

```
## All validation checks passed.
```